**Your skin - a touch screen?**
SkinTrack turns your skin into a touch interface for smart phones.
**http://dgit.in/skintrack**

**Facebook 360**
As Facebook rolls out VR, it also introduces 360 photos, instead of panoramic pictures.
**http://dgit.in/360Facebookx**

Every month we recognise the best article written by our community members and publish it here. Keep up the good work, Digitians!

# Securing your website 101

There are numerous ways to code websites that do exactly the same thing but is any of that code efficient and secure also?

*Praneet Sah*

Ever wondered how a hacker can get unauthorised access to websites? In easy words, by exploiting vulnerabilities. And guess what, it's you, the programmer, who creates these vulnerabilities and overall making it easier for any bad guy to hack into your website. Now this may sound ridiculous that how the programmer can be at fault, after all he's the person burning candle from both the ends, but imagine a scenario in which you are asked to create a website that includes a user dashboard, which allows user to update his information and profile picture, along with a button to delete the profile. Now, before we even begin to roll, you must be comfortable in coding a website with above specifications and able to convert pseudo code into real code on your own, in your choice of language.

A website like that would be easy to make but here's the golden rule, achieving functionality just means that you're done with 20% of the work. The rest 80% is all about optimising the code. Let me give you an example, imagine you are supposed to make a program that takes in name and phone number of 100 people and then it displays them. At one point in history, there would have been a time where we didn't have any loops and people would end up writing same input and output statements for 100 times with 100 odd variables. And then someone, frustrated with the repetition, created the concept of looping and now we can wrap up those hundreds of line into a mere five lines of code. Do you see a point? Both the programs can get the work done but the second program is efficient too.

Getting back to the point where we started, which was about creating a user dashboard. Here we have multiple ways to tackle the task and here's what you're expected to achieve (assume the user is already logged in):

1. Create a HTML form with relevant fields (including profile picture upload field)
2. Form will also display the current values that the fields hold
   a. If user clicks on submit: Put updated form's details in database and shift the uploaded file to server after renaming it to user's ID. Also redirect back to user dashboard.
   b. If user clicks on 'Delete profile' then delete the user from database using POST method.

That looks pretty easy and if you implemented it the same way, then take a minute and think that what exactly is wrong with that code. And yes, that pseudo code will get the work done.

Here actually lies a severe vulnerability because there's totally no validation of user's input and we're expecting the user to go along the pattern that we want him to. Unfortunately, not every user will have the same pattern and some (a.k.a. hackers) will try to mend it. Think from a hacker's (which mostly consists of script kiddies because real ones are busy printing money) point of view and you'll realize that following are the most commonly exploited vulnerabilities which arise due to poorly written code.

## Checking the uploaded file's extension

You asked the user to upload an image and the user uploaded a file named "DSC_002.jpg". He then checks the URL of uploaded image which is www.example.com/image/id_87.jpg (remember: you programmed it to rename the files to user's ID). Just to be double sure, the user uploads another image named "DSC_005.jpg" and again checks the URL which turns out to be same because it's programmed to renamed. So the user can figure out URL of any of his uploaded profile picture and he will now try to upload a shell file (it's like a backdoor access to servers; somewhat like a cPanel


within a PHP file). Shell files end with '.php' extension and since we aren't checking for the extension of files, this will get uploaded to "../image/id_87.php". The bad guy just needs to visit that address and instead of an image, he'll get a shady control panel which offers everything to take over your website. He'll have access to all your files which can even reveal the connection details of your database. So that's how a single user can steal all other user's details. To safeguard your website from such attacks, you need to check extension